Architecture Web Distribuée

GeoRisk Habitat



Assurance habitation et risque d'inondation

Présenté par

Groupe 6 · 4éme Actuariat

Les Membres



MOLKA BACCARI



FETHI MEJRI



MARIEM AMDOUNI



TAHA YASSINE SNOUSSI

Sommaire

01 Idée et problématique
02 Fonctionnalités
03 Architecture globale
04 Sécurité
05 Communications
06 Microservices et données
07 Technologies
08 Répartition des tâches
09 Git et gestion de projet
10 Planning et risques
11 Partie individuelle



Les inondations et autres événements climatiques causent chaque année des dégâts importants aux habitations.



Toutes les zones géographiques ne présentent pas le même niveau de risque.



L'assurance intervient principalement après le sinistre, au lieu de l'anticiper.

Comment évaluer précisément les risques climatiques pour mieux protéger les habitations et proposer une tarification plus juste ?

Une solution intelligente d'assurance habitation qui combine évaluation, prédiction et prévention des risques.

Trois piliers



Tarifer

La prime suit le score de risque de la zone (0 à 100) et les caractéristiques du bien.



Anticiper

Prévisions météo → probabilité d'inondation par zone → alerte aux assurés concernés.



Apprendre

Chaque alerte est comparée au résultat réel pour mesurer la qualité des prévisions.

1

seul aléa couvert :

l'inondation

Un parcours complet

De bout en bout

Produit → devis → contrat → alerte → sinistre → indemnisation simulée.

10 zones prédéfinies

De Tunis Centre à Sfax, chacune avec son score de risque.

Hors V1

Incendie, séisme, grêle : le champ hazard permet de les ajouter.

Les acteurs

OWNER

Propriétaire : biens, devis, contrat, alertes de sa zone, sinistres.

INSURER
EXPERT

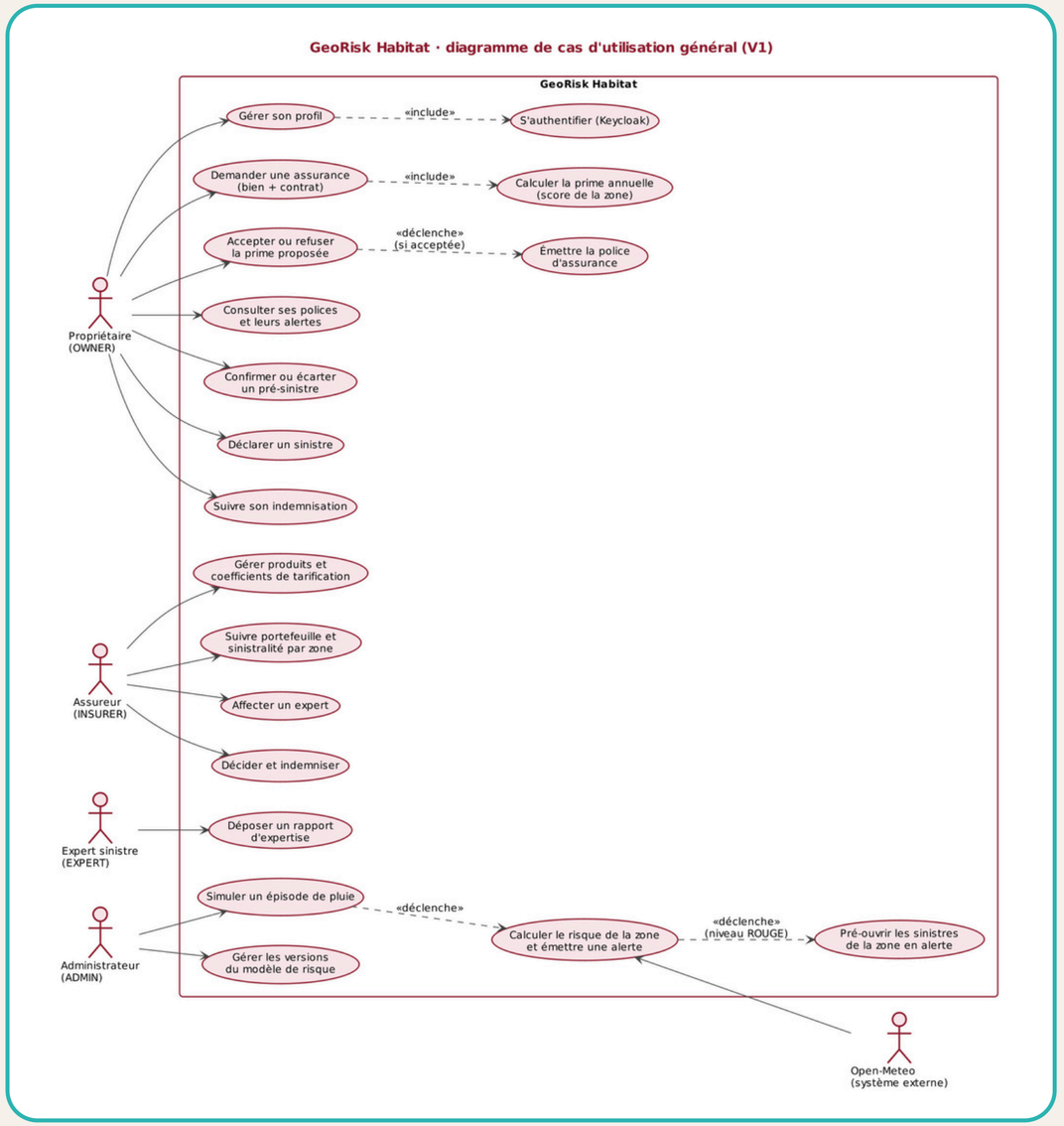
Assureur : produits et indemnisations.
Expert : rapports d'expertise.

ADMIN
SERVICE

Admin : modèle de risque et simulation.
Service : appels entre services.

Cas d'utilisation

Enregistrer un bien
Obtenir un devis
Souscrire un contrat
Recevoir une alerte
Déclarer un sinistre
Instruire et indemniser



Principales fonctionnalités

1. Enregistrer un bien et voir sa zone de risque sur la carte (MS1, MS5).
2. Obtenir un devis détaillé, puis souscrire un contrat (MS1, MS3).
3. Être alerté avant une inondation ; contrats signalés (MS5, MS2).
4. Gérer le sinistre : pré-sinistre, expertise, indemnisation (MS4).

Pourquoi des microservices ?

1. Polyglotte : le moteur de risque en Python, le métier en Spring Boot.

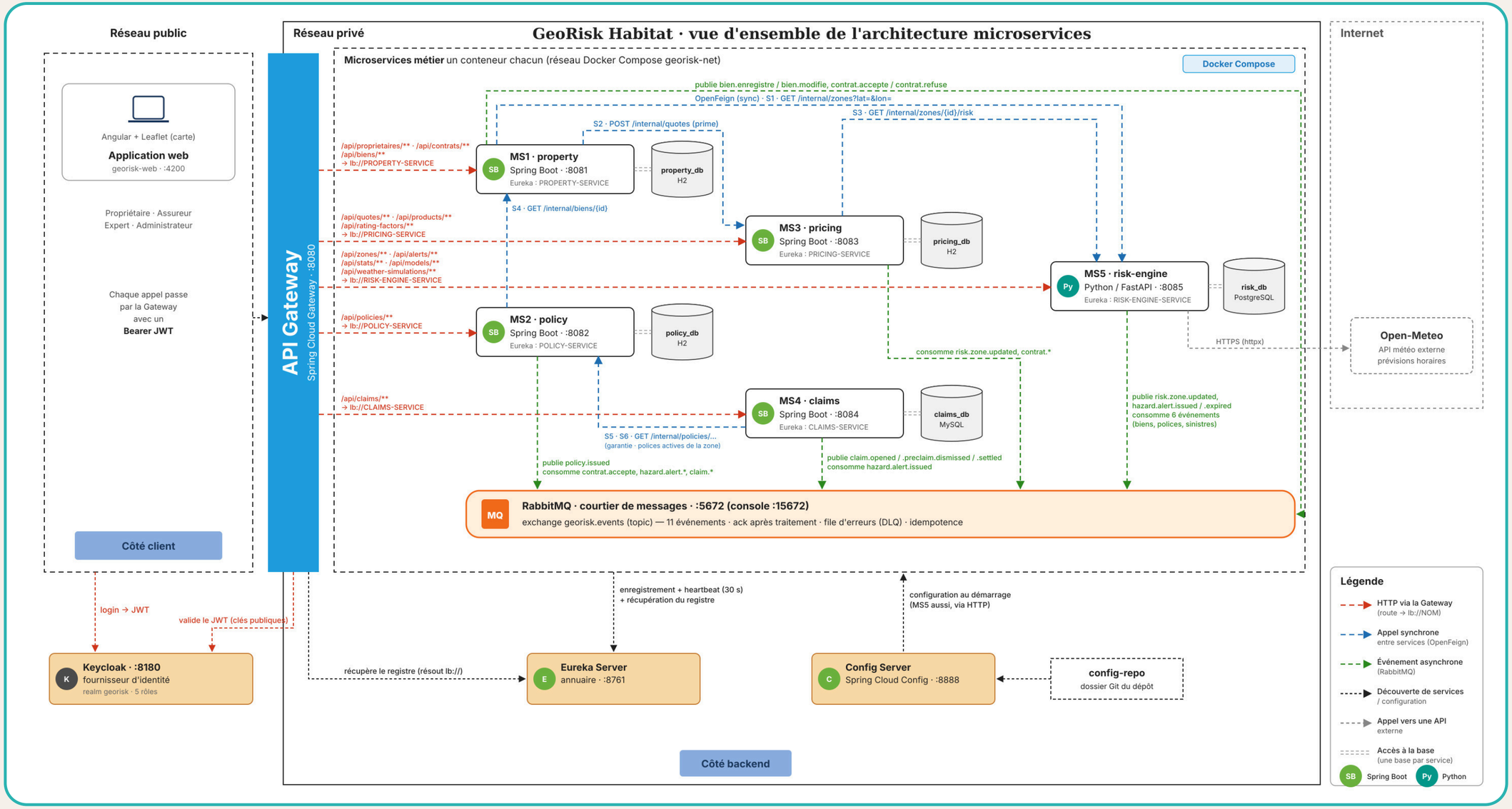
2. Charge ciblée : lors d'une alerte, on ne réplique que Policy et Claims.

3. Pannes isolées : MS5 en panne, les devis continuent avec le dernier score connu.

4. Un étudiant = un service = un contexte métier : chacun avance seul.

Architecture globale

- API Gateway : seul point d'entrée, vérifie le JWT
- Eureka : découverte des services
- Config Server : configuration centralisée
- Keycloak : comptes, rôles, jetons
- RabbitMQ : événements asynchrones
- Docker Compose : tout démarre en une commande
- /internal/** jamais exposé par la Gateway


GeoRisk Habitat · vue d'ensemble de l'architecture microservices
Réseau public
Réseau privé
Internet
Angular + Leaflet (carte)
Application web
georisk-web · :4200
Propriétaire · Assureur
Expert · Administrateur
Chaque appel passe par la Gateway avec un Bearer JWT
Côté client
API Gateway
Spring Cloud Gateway · :8080
Microservices métier un conteneur chacun (réseau Docker Compose georisk-net)
Docker Compose
publie bien.enregistre / bien.modifie, contrat.accepte / contrat.refuse
OpenFeign (sync) · S1 · GET /internal/zones?lat=&lon=
S2 · POST /internal/quotes (prime)
S3 · GET /internal/zones/{id}/risk
S4 · GET /internal/biens/{id}
/api/proprietaires/** · /api/contrats/**
/api/biens/**
→ lb://PROPERTY-SERVICE
MS1 · property
Spring Boot · :8081
Eureka : PROPERTY-SERVICE
property_db
H2
/api/quotes/** · /api/products/**
/api/rating-factors/**
→ lb://PRICING-SERVICE
MS3 · pricing
Spring Boot · :8083
Eureka : PRICING-SERVICE
pricing_db
H2
/api/zones/** · /api/alerts/**
/api/stats/** · /api/models/**
/api/weather-simulations/**
→ lb://RISK-ENGINE-SERVICE
MS5 · risk-engine
Python / FastAPI · :8085
Eureka : RISK-ENGINE-SERVICE
risk_db
PostgreSQL
/api/policies/**
→ lb://POLICY-SERVICE
MS2 · policy
Spring Boot · :8082
Eureka : POLICY-SERVICE
policy_db
H2
HTTPS (httpx)
Open-Meteo
API météo externe
prévisions horaires
consomme risk.zone.updated, contrat.*
/api/claims/**
→ lb://CLAIMS-SERVICE
MS4 · claims
Spring Boot · :8084
Eureka : CLAIMS-SERVICE
claims_db
MySQL
S5 · S6 · GET /internal/policies/...
(garantie · polices actives de la zone)
publie risk.zone.updated, hazard.alert.issued / .expired
consomme 6 événements (biens, polices, sinistres)
publie policy.issued
consomme contrat.accepte, hazard.alert.*, claim.*
publie claim.opened / .preclaim.dismissed / .settled
consomme hazard.alert.issued
MQ
RabbitMQ · courtier de messages · :5672 (console :15672)
exchange georisk.events (topic) — 11 événements · ack après traitement · file d'erreurs (DLQ) · idempotence
login → JWT
valide le JWT (clés publiques)
Keycloak · :8180
fournisseur d'identité
realm georisk · 5 rôles
enregistrement + heartbeat (30 s)
+ récupération du registre
configuration au démarrage
(MS5 aussi, via HTTP)
récupère le registre (résout lb://)
Eureka Server
annuaire · :8761
Config Server
Spring Cloud Config · :8888
config-repo
dossier Git du dépôt
Côté backend
Légende
HTTP via la Gateway
(route → lb://NOM)
Appel synchrone
entre services (OpenFeign)
Événement asynchrone
(RabbitMQ)
Découverte de services
/ configuration
Appel vers une API
externe
Accès à la base
(une base par service)
Spring Boot
Python

Sécurité

Keycloak · JWT

- Realm georisk, 5 rôles : OWNER, INSURER, EXPERT, ADMIN, SERVICE.
- Le frontend obtient un jeton JWT, envoyé à chaque requête (sans état).

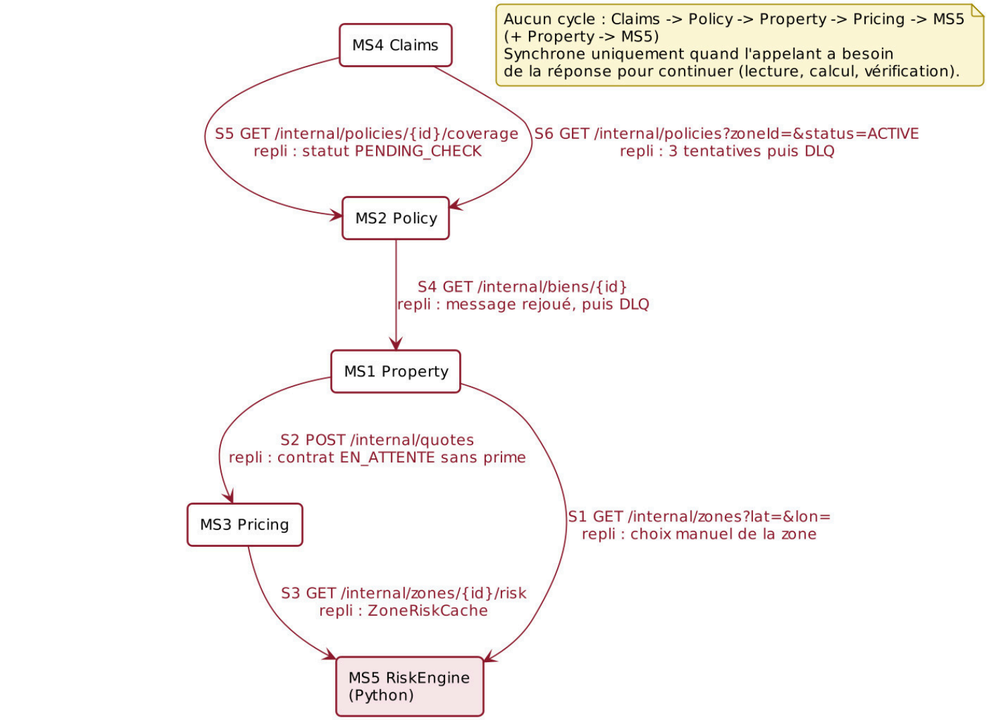
Contrôle d'accès

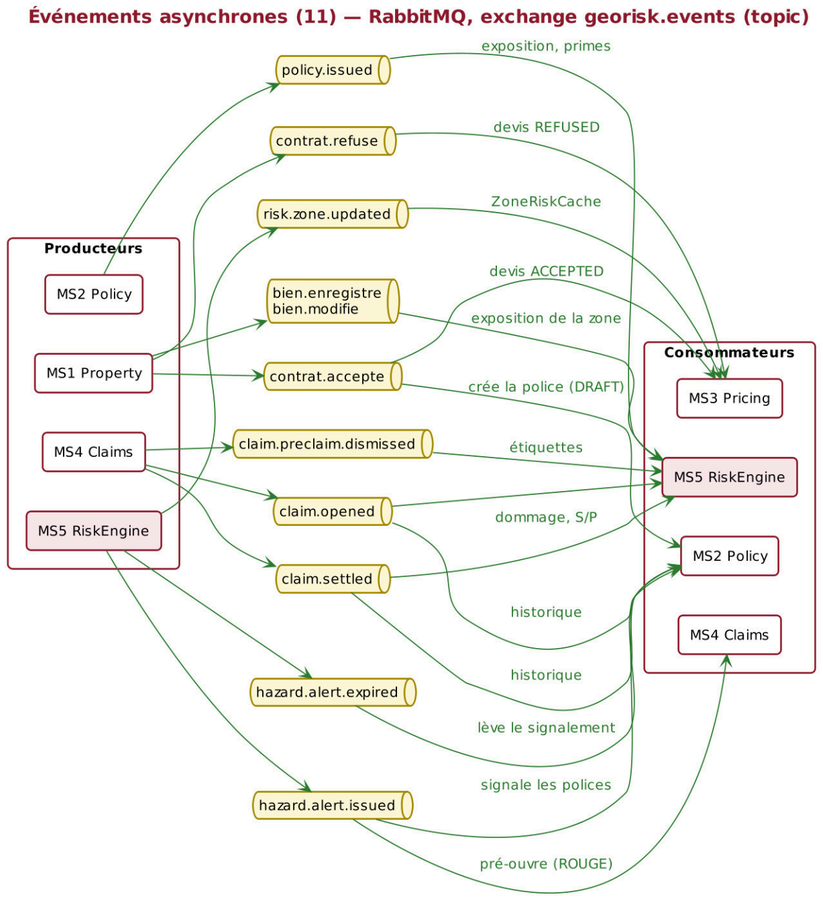
- La Gateway vérifie le jeton, chaque service le revérifie.
- Un propriétaire ne voit que ses objets (sinon 403).
- /internal/** : rôle SERVICE, jamais exposé.

Synchrone

Synchrone

- OpenFeign + Eureka, quand l'appelant a besoin de la réponse.
- 6 appels sans cycle, protégés par timeout + Resilience4j et une solution de repli.





Asynchrone

Asynchrone

- RabbitMQ, exchange topic georisk.events : on annonce un fait.
- 11 événements
- Ack après traitement, file d'erreurs (DLQ), idempotence.

4 + 1 microservices

MS1 · Property

Biens et contrats

H2 · port 8081

MS2 · Policy

Polices et garanties

H2 · port 8082

MS3 · Pricing

Produits, devis et primes

H2 · port 8083

MS4 · Claims

Sinistres et indemnisation

MySQL · port 8084

Un métier, un service

	Métier de l'assureur	Question à laquelle il répond	Ses données (sa propre base)
MS1 Property	**Souscription**	*Qui est le client, que veut-il assurer, accepte-t-il le prix ?*	Propriétaires, biens, contrats
MS2 Policy	**Gestion des polices**	*Qu'est-ce qui est couvert, et jusqu'à quand ?*	Polices, garanties, alertes
MS3 Pricing	**Tarification (actuariat)**	*Combien ça coûte ?*	Produits, coefficients, devis
MS4 Claims	**Gestion des sinistres**	*Que s'est-il passé, et combien rembourser ?*	Sinistres, expertises, paiements
MS5 RiskEngine	**Analyse du risque**	*Quel est le risque de la zone, et faut-il alerter ?*	Zones, météo, prédictions
À retenir	Comme chez un vrai assureur : chaque métier a son équipe, ses règles et ses données.	Frontière MS1 → MS2 : l'événement contrat.accepte, sans cycle synchrone.	Aucune base partagée : un service ne lit jamais la base d'un autre.

Une base par service

	H2	MySQL	PostgreSQL
MS1 Property	✓		
MS2 Policy	✓		
MS3 Pricing	✓		
MS4 Claims		✓	
MS5 RiskEngine			✓
Pourquoi ce choix ?	H2 (MS1, MS2, MS3) : données simples et peu volumineuses, base embarquée, rien à installer.	MySQL (MS4) : montants indemnisés à conserver durablement, base serveur, transactions ACID.	PostgreSQL (MS5) : agrégations par zone, colonnes JSON, séries de pluie, très bon support Python.

Technologies choisies

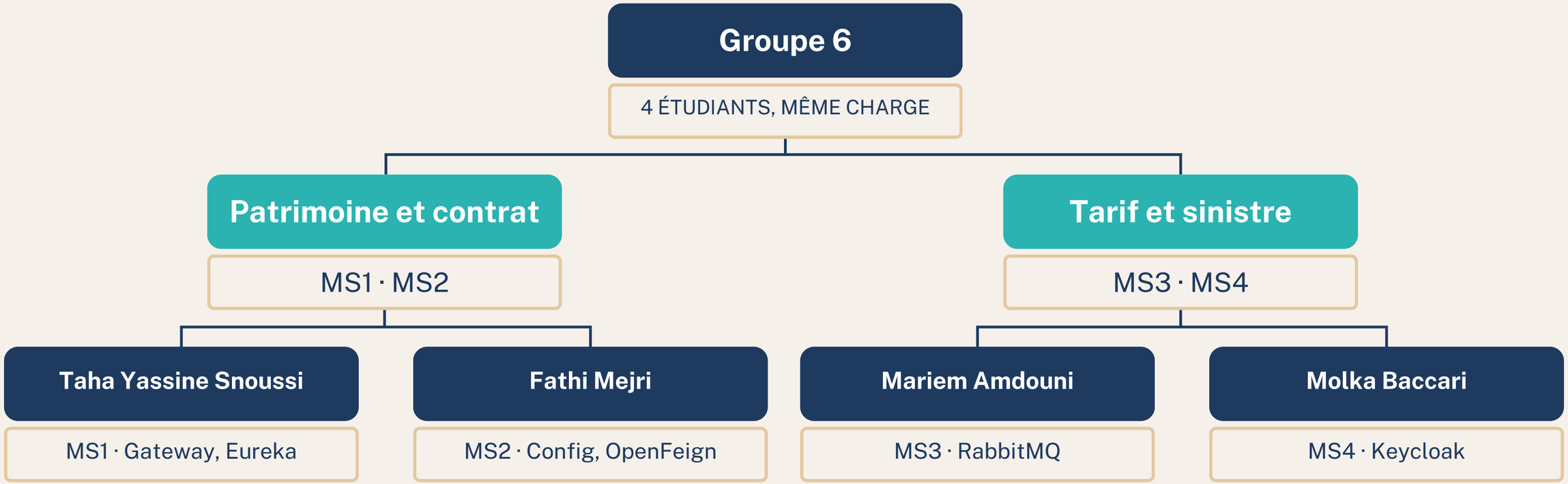
Back-end

- Java 17, Spring Boot 3
- Spring Cloud (Gateway, Eureka, Config Server)
- OpenFeign + Resilience4j
- RabbitMQ, Keycloak (JWT)

Front et données

- MS5 : Python FastAPI
- Angular + Leaflet
- H2, MySQL, PostgreSQL
- Docker Compose
- GitHub et GitHub Projects

Répartition des tâches



+ chacun : un module de MS5 et un écran

Dépôt Git et méthode

Structure

- Dépôt public GeoRisk-4Actuariat-g6, enseignante collaboratrice
- Backend/ (un projet par service)
- Frontend/, Documentation/, README

Méthode

- Une branche par service (property-service, pricing-service…)
- Fusion dans master par pull request
- Chacun commite avec son compte
- Suivi sur GitHub Projects

Notre planning

S4

Validation 1 : conception, contrats d'API figés

S5 – S6

Gateway, Eureka, Config, OpenFeign, bouchon MS5

S7 – S8

RabbitMQ, Keycloak, alertes MS5

S9 – S10

Docker Compose, frontend, tests

Validation 2

Démo : scénarios A et B

2

risques identifiés, chacun sa parade

Un service en retard

Contrat figé, bouchon MS5, données de test.

Météo calme le jour J

Simulation de pluie pour la démonstration.

Taha Yassine Snoussi

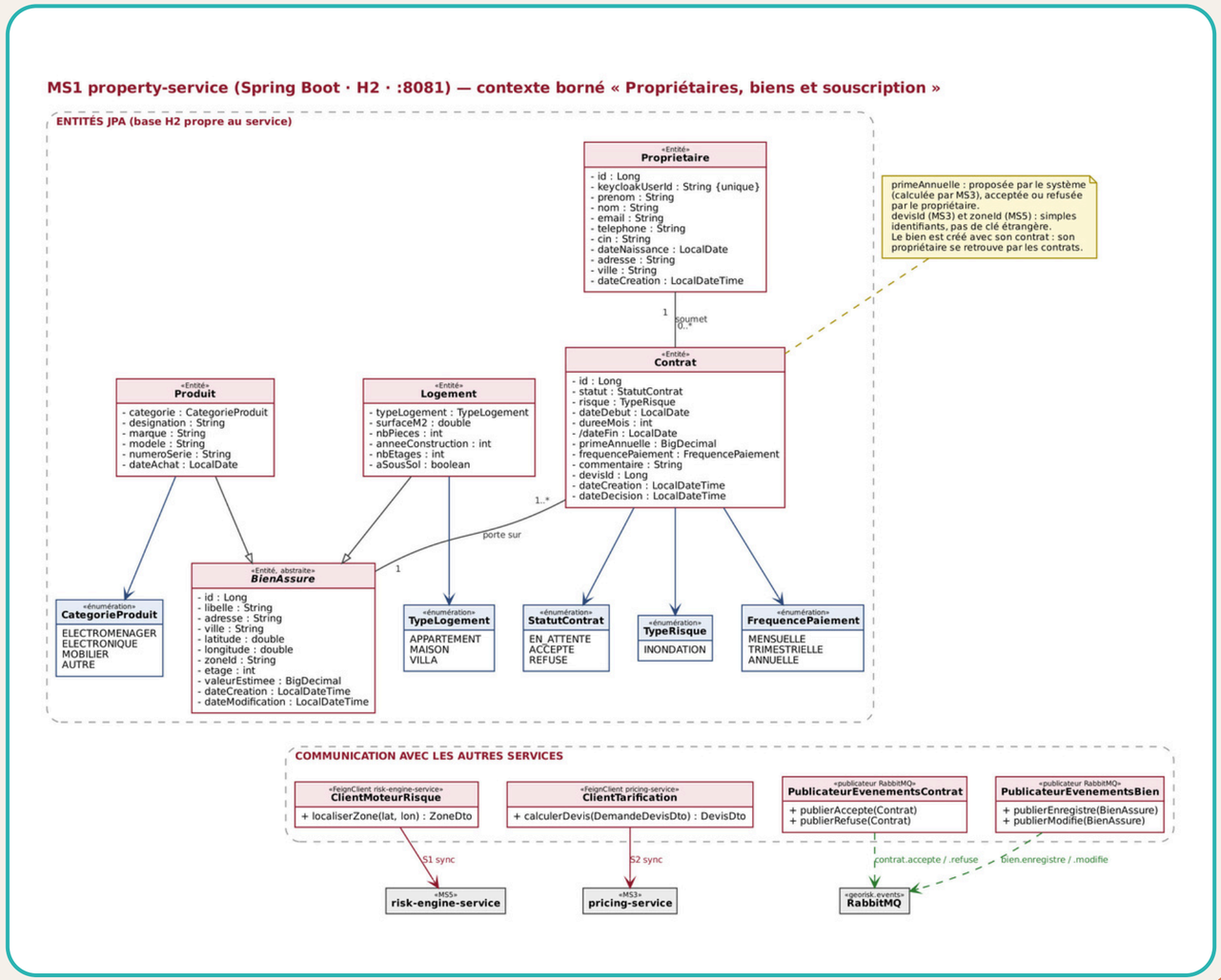
Property

Souscription : propriétaires, biens et contrats ; le propriétaire accepte ou refuse la prime proposée.

API REST
POST /api/contrats → 201 · 400 · 404
PATCH /api/contrats/{id} → 200 · 400 · 403 · 404 · 409
GET /internal/biens/{id} → 200 · 404

Synchrone (OpenFeign)
→ MS5 (S1) : zone du bien (repli : choix manuel)
→ Pricing (S2) : prime du contrat (repli : sans prime)
← appelé par Policy (S4)

Asynchrone (RabbitMQ)
Publie bien.enregistre, bien.modifie → MS5
Publie contrat.accepte → MS2, MS3 ; contrat.refuse → MS3



Fethi Mejri

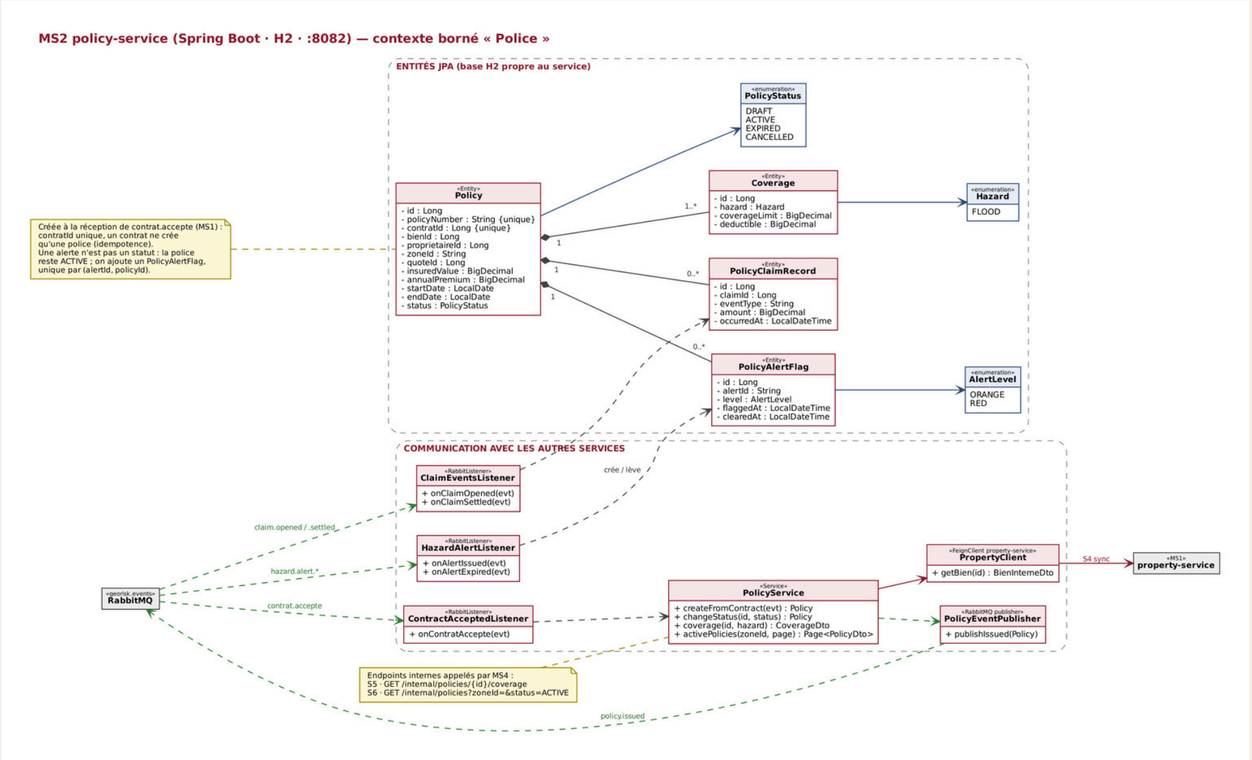
Policy

Polices et garanties ; signale les polices touchées par une alerte.

API REST
PATCH /api/policies/{id} → 200 · 403 · 404 · 409
GET /api/policies/me/alerts → 200
GET /internal/policies/{id}/coverage → 200 · 404
GET /internal/policies?zoneId=&status=ACTIVE → 200

Synchrone (OpenFeign)
→ Property (S4) : valeur et zone du bien
← appelé par Claims (S5, S6)

Asynchrone (RabbitMQ)
Consomme contrat.accepte (crée la police), hazard.alert.*, claim.opened, claim.settled
Publie policy.issued



Mariem Amdouni

Pricing

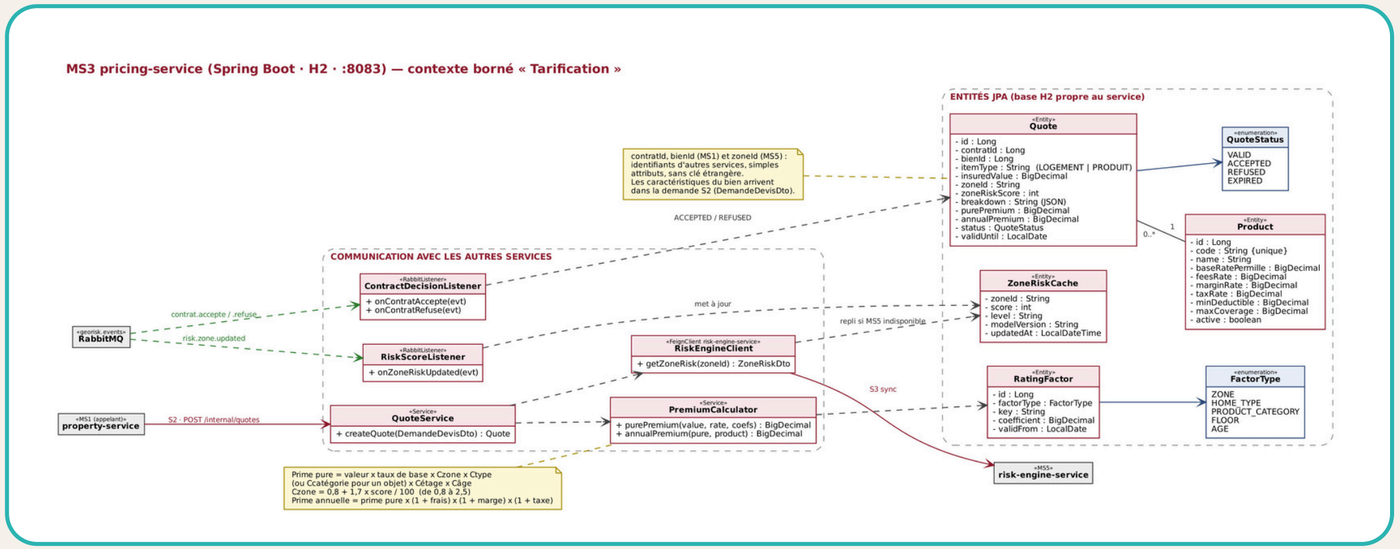
Produits, coefficients, devis : du bien à la prime.

API REST
GET /api/quotes/{id} → 200 · 404
PUT /api/rating-factors/{id} → 200 · 400 · 404
POST /internal/quotes → 201 · 400 · 404

Synchrone (OpenFeign)
← appelé par Property (S2) : prime du contrat
→ MS5 (S3) ; panne : dernier score en cache

Asynchrone (RabbitMQ)
Consomme risk.zone.updated, contrat.accepte / contrat.refuse

Prime = valeur × taux × Czone × Ctype × Cétage × Câge



Molka Baccari

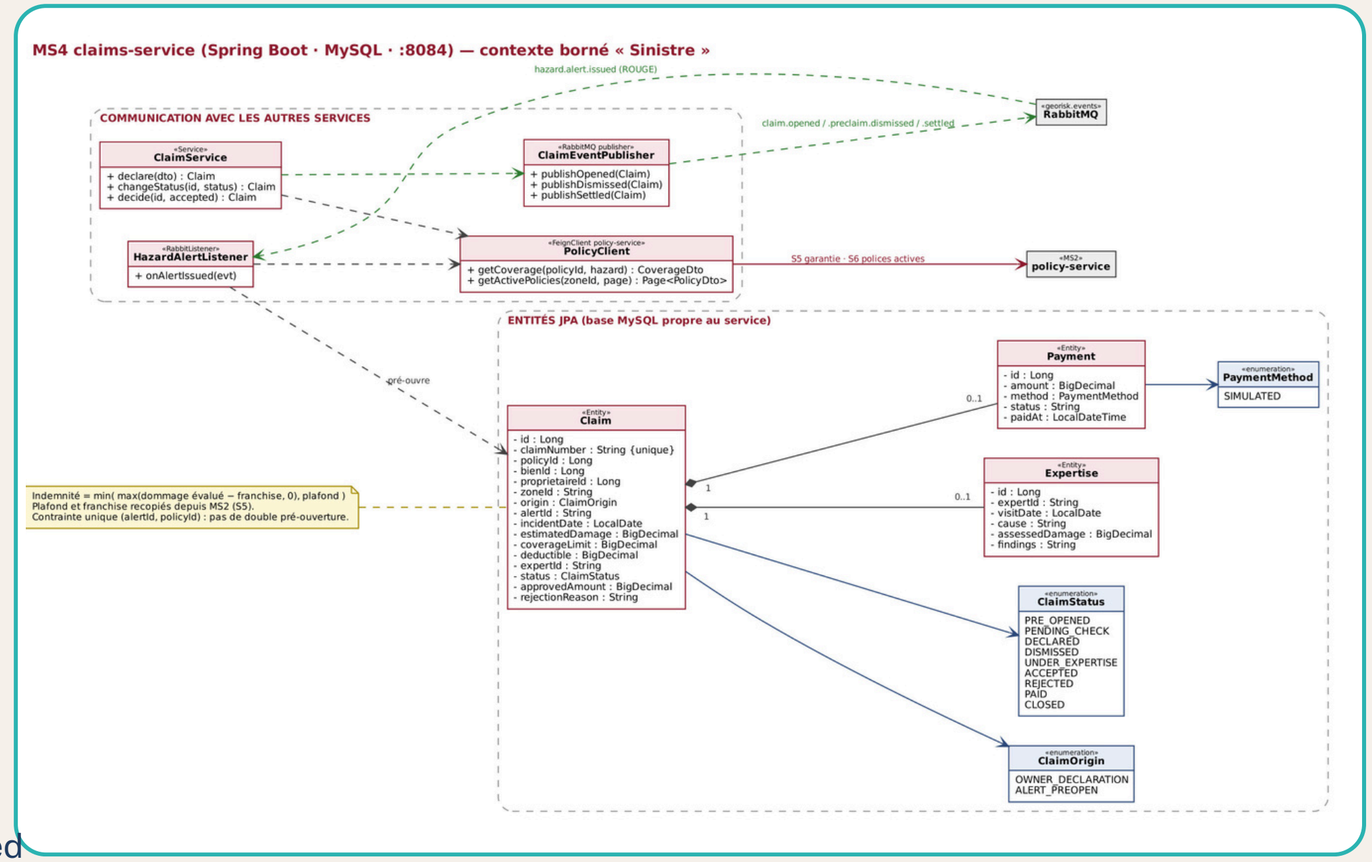
Claims

Du pré-sinistre à l'indemnisation.

API REST
POST /api/claims → 201 · 400 · 409
PATCH /api/claims/{id} → 200 · 403 · 404 · 409
POST /api/claims/{id}/expertise → 201 · 400 · 403 · 409
POST /api/claims/{id}/decision → 201 · 400 · 409

Synchrone (OpenFeign)
→ Policy : garantie (S5), polices actives (S6)
Panne : sinistre en PENDING_CHECK

Asynchrone (RabbitMQ)
Consomme hazard.alert.issued (pré-sinistre si ROUGE)
Publie claim.opened, claim.preclaim.dismissed, claim.settled



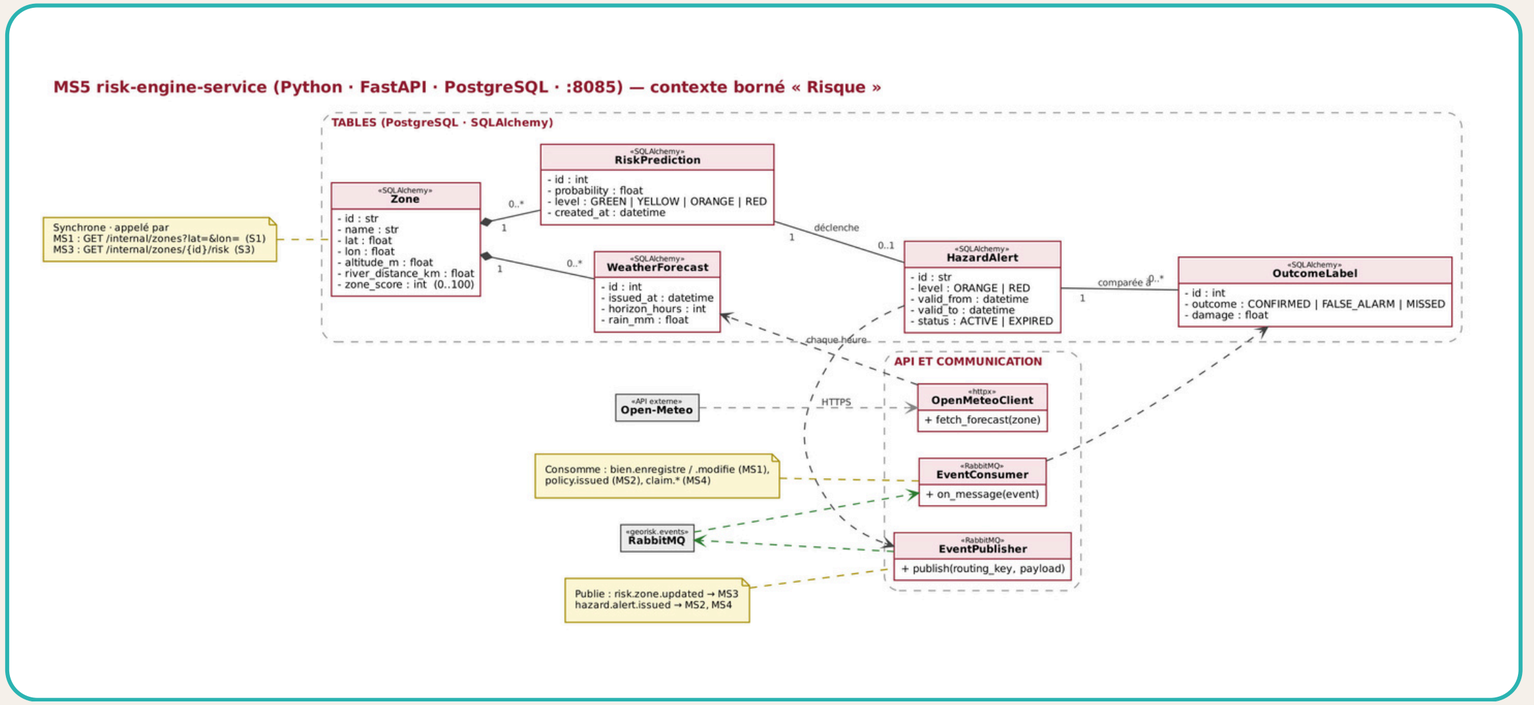
RiskEngine

Microservice commun : un module par étudiant (zones, météo, modèle, alertes).

API REST
GET /api/zones/{id}/risk → 200 · 404
POST /api/weather-simulations → 201 · 400 · 404
GET /internal/zones?lat=&lon= → 200 · 400
GET /internal/zones/{id}/risk → 200 · 404

Synchrone
← appelé par Property (S1) et Pricing (S3) via Eureka

Asynchrone (RabbitMQ)
Publie risk.zone.updated, hazard.alert.issued / expired
Consomme 6 événements (biens, contrats, sinistres)



Merci

Vos questions ?

